

The present work was submitted to the Chair of Computational Science and Engineering.

presented by:

Enes Çağlar Korkmazgöz

464051

Simulation Sciences Seminar Project Report

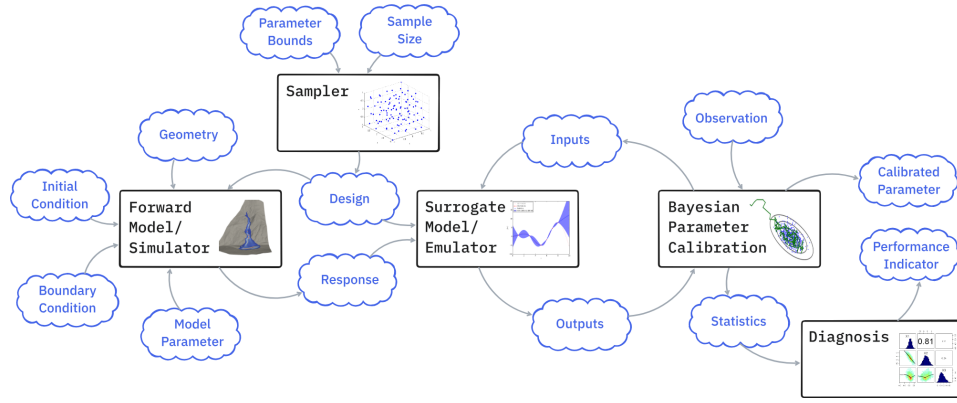
Benchmarking of Sampling Methods for Probabilistic Calibration in Gravity-Driven Mass Flow Modeling

Supervisors :

Alan Correa

Mithlesh Kumar

Aachen, 31.07.2026



Benchmarking of Sampling Methods for Probabilistic Calibration in Gravity-Driven Mass Flow Modeling

Enes Çağlar Korkmazgöz

ABSTRACT

Bayesian calibration of gravity-driven mass flow models requires repeated exploration of a posterior distribution and may depend strongly on the selected sampling method. This report presents `mcmc-bench`, a modular framework that integrates Random-Walk Metropolis-Hastings, `emcee`, `dynesty`, PyMC Slice, and PyMC Sequential Monte Carlo through Nextflow and UM-Bridge. All methods calibrate the Voellmy friction parameters μ and ξ using the same RobustGaSP surrogate, observations, priors, and likelihood. The samplers produced similar posterior means and standard deviations in the case study. In the single benchmark run, `dynesty` required the fewest surrogate-model evaluations and achieved the highest normalized efficiency. The results demonstrate that sampler-specific software environments can be integrated into a reusable workflow while maintaining a common statistical target and output format. The performance findings remain specific to the selected problem, configurations, and execution environment.

DECLARATION OF OWN WORK

I hereby declare that this thesis has been written independently and that no sources other than those cited have been used. All passages taken verbatim or paraphrased from published or unpublished works have been clearly marked as such. This work has not been submitted, in whole or in part, for any other academic qualification.

DECLARATION OF AI TOOL USAGE

The following AI tools were used during the preparation of this thesis:

- **ChatGPT 5.6 Sol** — This assistant was used for grammar checking of the report and debugging of the source code throughout the project. The author takes full responsibility for the accuracy, interpretation, and final content of this report.
- **Google Scholar LABS** — This tool was used to assist in the literature review process by providing relevant academic papers and references.

All AI-assisted content has been reviewed and verified by the author. No AI tool was used to generate results, draw conclusions, or replace independent scientific reasoning.

Aachen, 31.07.2026

Enes Çağlar Korkmazgöz

CONTENTS

LIST OF FIGURES	IX
LIST OF TABLES	XI
1 INTRODUCTION	1
1.1 Motivation	2
1.2 Research Questions	4
1.3 Outline	4
2 BACKGROUND AND RELATED WORK	5
2.1 The Inverse Problem	5
2.2 The Bayesian Approach to the Inverse Problem	6
2.3 The Bayesian Calibration Pipeline	8
2.4 Sampling Methods for Bayesian Inference	9
3 BENCHMARKING SAMPLING METHODS	11
3.1 Implementation Details	11
3.2 Sampler Integration	12
3.2.1 Random-Walk Metropolis-Hastings	12
3.2.2 emcee	12
3.2.3 dynesty	13
3.2.4 PyMC Slice and Sequential Monte Carlo	13
4 EVALUATION	15
4.1 Experimental Setup	15
4.2 Metrics	16
4.3 Results	18
4.3.1 Posterior Agreement	18
4.3.2 Convergence Diagnostics	18
4.3.3 Computational Efficiency	20
4.4 Discussion	20

Contents

5	SUMMARY AND FUTURE WORK	23
5.1	Summary	23
5.2	Future Work	23
	BIBLIOGRAPHY	25

LIST OF FIGURES

1.1	Topography and release mass distribution of the 2017 Bondo landslide, together with the simulated impact area and deposit distribution for Voellmy parameters $\mu = 0.23$ and $\zeta = 1000 \text{ m s}^{-2}$. Reproduced without modification from Zhao and Kowalski [32, Fig. 2] under the CC BY 4.0 license.	3
2.1	Relationship between the forward and inverse problems.	5
2.2	General workflow of Bayesian calibration.	8
4.1	Joint posterior distributions obtained for (a) RWMH, (b) emcee, (c) PyMC Slice, (d) PyMC SMC, and (e) dynesty.	19

LIST OF TABLES

2.1	Selected state-of-the-art sampling algorithms.	9
4.1	Sampler configurations used in the benchmark run.	16
4.2	Posterior summary statistics from the single benchmark run.	18
4.3	Convergence diagnostics from the single benchmark run.	18
4.4	Computational-efficiency results from the benchmark run.	20

1 INTRODUCTION

The source code, workflow definitions, and configuration files of this study are publicly available in the [mcmc-bench GitHub repository](#).

Computational models are widely used to represent and predict the behaviour of complex systems, but their predictions often depend on input parameters whose values are unknown for the application under study. Model calibration uses observations of the physical system to learn about these unknown inputs by adjusting the corresponding model parameters. This connection between observations and model parameters is important not only for improving predictive performance, but also for representing the uncertainty that remains after calibration and accounting for discrepancies between the simulator and reality [15].

Bayesian inference provides a formal framework for *learning from experience*. Existing knowledge or assumptions about unknown quantities are represented by a prior distribution, while observations enter the analysis through the likelihood. Combining these two sources of information produces the posterior distribution, which expresses what has been learned after observing the data [33]. This principle is not restricted to a particular application area; Bayesian methods are widely used to reason about uncertain parameters and predictions across a broad range of scientific disciplines [23]. In model calibration, the posterior therefore describes which parameter values remain plausible after seeing the model with observations, rather than returning only a single fitted value [15].

All of the model parameters do not have the same interpretation. Physical parameters have meaning within the scientific description of a system, although their values may be unknown or difficult to measure directly from the physical system. On the other hand, tuning parameters have no direct physical interpretation; they are often introduced as simplified representations of processes that the simulator does not explicitly resolve and are adjusted to improve agreement with observations [4]. This distinction affects how calibration results should be interpreted: learning about a physical parameter may provide information about the system, whereas an inferred tuning parameter remains specific to the model. In the Voellmy model

considered here, μ and ζ enter the resistance law as dry-Coulomb and turbulent friction coefficients, respectively. However, they are effective, conceptual parameters rather than directly measurable material constants, and their values are commonly inferred by back-analysing observed events from the field data [13, 31].

Bayesian calibration typically requires repeated evaluations of the forward model, which might be expensive. A statistical emulator can be used to approximate the simulator outputs from a smaller set of precomputed runs; RobustGaSP provides Gaussian-process emulators for this purpose [11, 31]. The benchmark in this report therefore operates on an existing RobustGaSP surrogate of the gravity-driven mass flow model rather than executing the full simulator during sampling. All integrated samplers use this same surrogate model and target the same sampler-agnostic posterior. Here, sampler-agnostic means that the prior and likelihood, and therefore the mathematical target distribution, are defined independently of the sampling algorithm. The benchmark can thus evaluate how accurately and efficiently each method approximates the same target, while recognizing that finite-run and implementation effects may still produce different numerical results.

1.1 MOTIVATION

Gravity-driven mass flow models simulate the runout of avalanches, debris flows, and related flow-like landslides, providing estimates of quantities such as flow depth, velocity, and impact area. These outputs points hazard maps and quantitative risk assessments and can inform mitigation planning and model-based decision support [31]. Their practical value, however, depends on the reliability of the simulated event behaviour. Because the effective resistance parameters cannot be measured directly, they must be calibrated against observations, and the uncertainty associated with this calibration must be considered when interpreting model predictions [13, 31].

A deterministic calibration can be formulated as an optimization problem that returns a parameter vector minimizing the discrepancy between model outputs and observations. However, observations contain measurement error. If these sources of error are not represented explicitly, the fitted parameters may include noise leading to biased conclusions [4, 15]. Calibration may also suffer from identifiability problems: different parameter combinations, or different allocations between parameters and model discrepancy, can explain the observations similarly well [2]. Consequently, a single best point does not account for the result. The relevant ques-

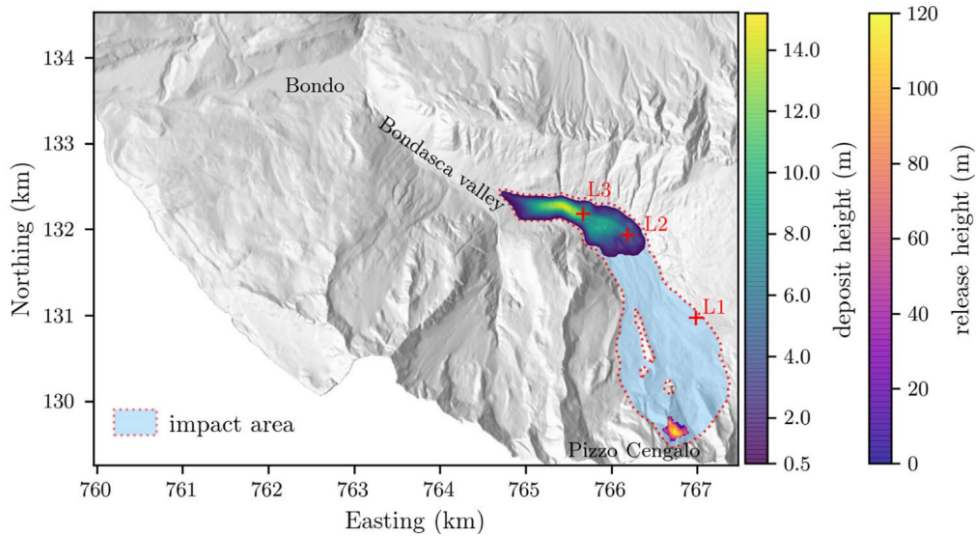


Figure 1.1: Topography and release mass distribution of the 2017 Bondo landslide, together with the simulated impact area and deposit distribution for Voellmy parameters $\mu = 0.23$ and $\zeta = 1000 \text{ m s}^{-2}$. Reproduced without modification from Zhao and Kowalski [32, Fig. 2] under the [CC BY 4.0 license](#).

tion is instead: *which combinations of parameter values are supported by the observations, and how is uncertainty distributed across those combinations?*

Bayesian calibration addresses this problem by representing uncertainty about model parameters through a posterior distribution rather than a single estimate. The prior has assumptions before observing the calibration data, while the likelihood describes how the observations relate to model outputs; together they define the posterior on the selected model and data [15]. Reliable predictive uncertainty quantification also requires consideration of model discrepancy and surrogate uncertainty [4, 31]. Such study is not evaluated in this benchmark; here, the posterior serves as the common target distribution for all of the samplers.

When posterior quantities cannot be computed analytically, sampling algorithms are used to explore the target distribution [1, Sections 1–2]. These samplers differ in how they handle challenging posterior geometries, and their relative accuracy and efficiency can depend on the structure of the target distribution and on implementation details [1, Abstract and Sections 1 and 4.1].

The samplers currently integrated into the benchmark use different Python packages and run in sampler-specific software environments. However, the `mcmc-bench` repository implements a modular and extensible benchmarking framework rather than a Python-only sampler interface. Its execution is orchestrated as a Nextflow

workflow, while UM-Bridge exposes the surrogate through a language-agnostic model interface. This separation allows future sampling methods implemented in other programming languages to be integrated without modifying the model interface [24, Summary and Statement of Need]. For the current comparison, all samplers are configured to use the same surrogate, prior, and likelihood, which together define the common sampler-agnostic posterior.

1.2 RESEARCH QUESTIONS

Our research questions are as follows:

- RQ1* How can different sampling methods be integrated into a modular, extensible, and reusable framework for Bayesian calibration of a surrogate-based gravity-driven mass flow model?
- RQ2* Which accuracy, convergence, and computational-efficiency criteria enable a meaningful comparison of the integrated sampling methods, and how do the samplers perform according to these criteria in the Voellmy calibration case study?

1.3 OUTLINE

The followings of this report are organized as follows. Chapter 2 introduces inverse problems, Bayesian calibration, surrogate modeling, and the selected sampling methods. Chapter 3 describes the common benchmarking framework and the integration of the five samplers. Chapter 4 presents the experimental setup, comparison criteria, results, and discussion. Finally, Chapter 5 summarizes the findings and outlines future work.

2 BACKGROUND AND RELATED WORK

2.1 THE INVERSE PROBLEM

Computational models take a set of inputs and produce quantities that can be observed; this defines the forward model. An inverse problem begins with observations and seeks to infer parameter values that are consistent with those observations under the model. In model calibration, this inference is performed by comparing simulator outputs with observed data. However, the inverse relationship need not be unique: multiple parameter combinations may explain the observations similarly, particularly in the presence of measurement uncertainty and discrepancies between the simulator and reality [2, 15]. Figure 2.1 illustrates the relationship between these two problems.

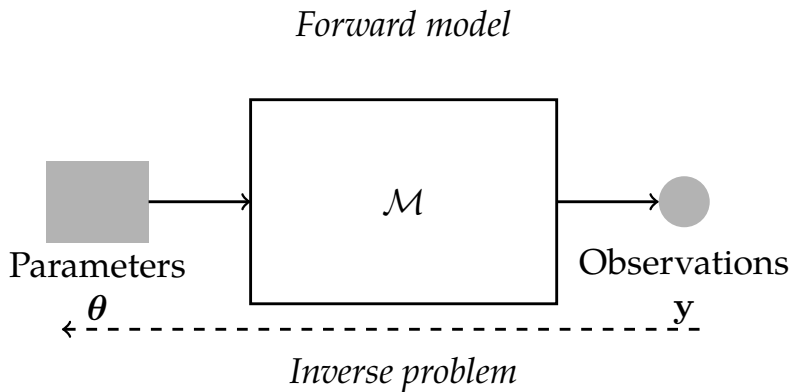


Figure 2.1: Relationship between the forward and inverse problems. The forward model maps parameters θ to predicted observables, whereas the inverse problem uses observations y to infer plausible parameter values. Adapted from the conceptual illustration by Moura Neto and da Silva Neto [17].

In inverse-problem theory, a problem is called ill-posed if a solution does not exist, is not unique, or changes strongly under small changes in the data [17, Chap. 3]. Calibration problems often problematic under the last two issues: because of mea-

surement noise, limited data, and model error, several different parameter combinations can fit the observations equally. Two common solutions are regularization and probabilistic methods, which add extra information about the unknown parameters [3, 27]. This study follows the probabilistic route. In gravity-driven mass flow modeling, the Bayesian calibration is often used, which estimates effective friction parameters from observed events and also quantifies the remaining uncertainty [13, 32].

2.2 THE BAYESIAN APPROACH TO THE INVERSE PROBLEM

Bayesian calibration treats the unknown model parameters as uncertain quantities and uses Bayes' theorem to update prior information with observed data [15, 33]. Let θ denote the model parameters and y the observations. The prior distribution $p(\theta)$ represents information about the parameters before considering y , while the likelihood $p(y | \theta)$ describes how compatible the observations are with a given parameter value. Bayes' theorem combines these components into the posterior distribution:

$$p(\theta | y) = \frac{p(y | \theta) p(\theta)}{p(y)}, \quad (2.1)$$

where $p(y)$ is the model evidence, which does not depend on θ and normalizes the posterior distribution.

The evidence can be written as $p(y) = \int p(y | \theta) p(\theta) d\theta$, but this integral is generally not available to us. Because $p(y)$ is constant with respect to θ , many MCMC methods can explore the posterior using only its unnormalized density [1, 15]:

$$p(\theta | y) \propto p(y | \theta) p(\theta). \quad (2.2)$$

In computation, the unnormalized posterior is commonly evaluated on the logarithmic scale. This converts the product into a sum and reduces numerical errors (in floats) when the probability densities are very small:

$$\log \tilde{p}(\theta | y) = \log p(y | \theta) + \log p(\theta), \quad (2.3)$$

where $\tilde{p}(\theta | y)$ denotes the unnormalized posterior density. Nested sampling differs from standard MCMC in this respect: it keeps the likelihood and prior separate in addition to the prior probability mass calculations and as a result estimates the model evidence, or in short terms, it naturally has the $d\theta$ term.

The likelihood connects the observations to the output of the forward model. The calibration framework uses an additive Gaussian residual model:

$$y_i = f_i(\theta) + \varepsilon_i, \quad \varepsilon_i \stackrel{\text{iid}}{\sim} \mathcal{N}(0, \sigma^2), \quad (2.4)$$

where $f_i(\theta)$ denotes the model prediction corresponding to observation y_i , and σ is the residual standard deviation. Under this assumption, the log-likelihood for n observations is

$$\log p(y \mid \theta, \sigma) = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - f_i(\theta))^2. \quad (2.5)$$

For the general formulation, $f_i(\theta)$ denotes the full forward-model response. In the benchmark implementation, it is replaced by the surrogate prediction $\hat{f}_i(\theta)$.

The residual term may combine the model error and measurement error. Consequently, independent Gaussian residuals are a modeling assumption rather than a property guaranteed by the observations [4, 13, 15].

In Bayesian calibration based on a full computational model, each likelihood evaluation generally requires a new forward simulation. If this simulation is expensive, a surrogate model can approximate the input–output relationship of the original forward model using a limited set of precomputed runs [11, 31]. This substitution substantially reduces the cost of repeated model evaluations, but introduces an additional approximation layer: the surrogate requires an initial training data set and may differ from the full-model response. Its accuracy and uncertainty must therefore be considered when interpreting the resulting posterior [11, 31, 32].

In the present benchmark, the likelihood is evaluated using an already constructed RobustGaSP surrogate rather than the full gravity-driven mass flow simulator. All samplers therefore target the posterior defined by the same surrogate, prior, likelihood, and observations. Although a surrogate evaluation is considerably cheaper than a full simulation, it is still performed repeatedly, and different sampling methods can require different numbers of evaluations and introduce different computational overheads. Runtime and resource use remain relevant comparison criteria, with greater computational demand also results in greater energy use [16].

2.3 THE BAYESIAN CALIBRATION PIPELINE

A Bayesian calibration pipeline connects the statistical components of the inverse problem in a defined sequence. The unknown parameters are assigned prior distributions, the forward model or its surrogate maps these parameters to predicted observables, and the likelihood compares these predictions with the calibration data. The prior and likelihood then define the posterior distribution, which a sampling algorithm explores to produce posterior samples [11, 15]. Keeping these components separate is important: changing the surrogate, prior, likelihood, or observations changes the target posterior, whereas changing the sampler changes the numerical strategy used to approximate that target.

Figure 2.2 provides a broader view of this process, including the construction of a surrogate from full-model simulations and the calibration and diagnostic stages. The present benchmark starts from an already trained surrogate and focuses on posterior sampling and sampler diagnostics.

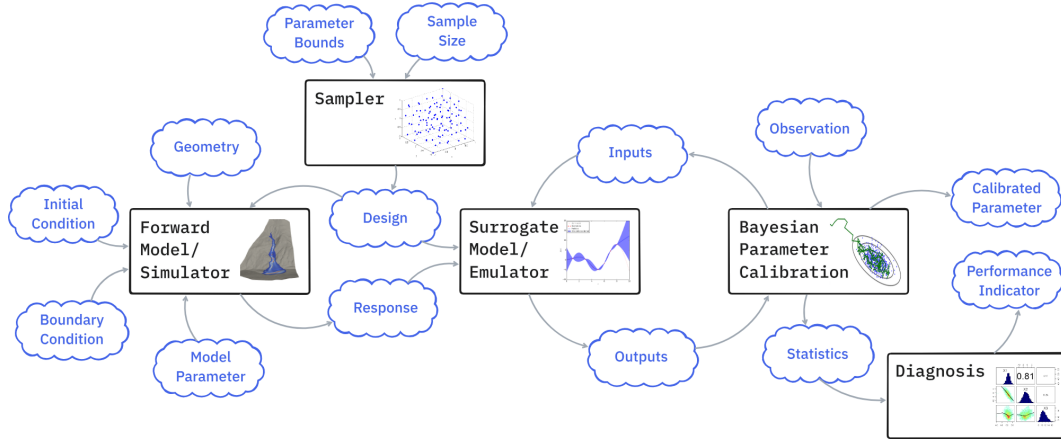


Figure 2.2: General workflow of Bayesian calibration. Illustration courtesy of Alan Correa.

The `mcmc-bench` framework implements this conceptual pipeline by separating model evaluation from sampler execution. The pre-trained, R-based Robust-GaSP surrogate is used through UM-Bridge, so sampler implementations interact with it through a common, language-agnostic interface rather than depending directly on the surrogate’s internal implementation [11, 24, 31]. Nextflow coordinates the sampler-specific software environments, and subsequent diagnostic and reporting steps as a reproducible computational workflow [7]. The currently integrated samplers use Python packages, but UM-Bridge allows future sampler implementations written in other programming languages to use the same model interface.

2.4 SAMPLING METHODS FOR BAYESIAN INFERENCE

Modern Bayesian computation includes gradient-based methods such as HMC and NUTS, ensemble and slice-based approaches, population methods such as Sequential Monte Carlo, nested-sampling algorithms, and more recent methods that use normalizing flows to construct proposals or transform the sampling space. Representative implementations use several programming languages and software packages, as summarized in Table 2.1.

Table 2.1: Selected state-of-the-art sampling algorithms.

Algorithm	Package(s)	Language(s)	Benefit(s)
HMC/NUTS [8, 14]	PyMC [21]	Python	Gradient-informed exploration
Affine-invariant ensemble [9, 10]	emcee	Python	Robustness to linear rescaling and correlation
Slice sampling [18]	PyMC [22]	Python	Adaptive interval construction
SMC [6]	PyMC [20]	Python	Population-based exploration
Nested sampling [5, 25, 26]	dynesty, UltraNest	Python	Posterior and evidence estimation
Flow-based nested sampling [29]	nessai	Python	Learned constrained-prior proposals
Flow-enhanced MCMC [30]	flowMC	Python	Learned global proposals for complex posteriors

Given the time constraints of the present work, the initial benchmark integrations focus on Python-based packages. This is an implementation-scope decision rather than a limitation of the underlying framework. The `mcmc-bench` benchmarking framework uses a Nextflow workflow to orchestrate sampler-specific environments, while UM-Bridge decouples the model interface from the sampler implementation. The framework can therefore be extended with sampler packages implemented in other languages without changing the surrogate-model interface [7, 24]. At present, however, the surrogate implementation provides model evaluations but not gradient information, so the current benchmark directly supports only gradient-free samplers.

Even though not all the samplers require gradient information or not, there is one thing in common. Calibration problems generally require numerical sampling because posterior expectations and credible intervals are not available analytically. Sampling methods differ in how they explore the target distribution: some construct local Markov-chain proposals, some use multiple interacting chains, and oth-

ers visit nested likelihood regions. Their performance can therefore vary with posterior geometry, dimensionality, tuning choices, and implementation details [1, Sections 1–2 and 4.1]. The present benchmark considers five implementations representing several of these approaches: Random-Walk Metropolis-Hastings (RWMH), emcee, dynesty, PyMC Slice, and PyMC Sequential Monte Carlo (SMC).

The selected methods represent different strategies for exploring the posterior. RWMH uses local random-walk proposals followed by an accept/reject decision [12], while emcee employs interacting walkers and affine-invariant moves to reduce sensitivity to linear correlations [9, 10]. Slice sampling constructs a region around the current state and repeats the process for each subsequently selected value [18]. Sequential Monte Carlo evolves and resamples a population of weighted particles through a sequence of intermediate distributions [6], whereas nested sampling successively explores constrained likelihood regions and additionally estimates the model evidence [25, 26]. These methodological differences lead to different tuning requirements and computational costs [1].

This chapter provides only the conceptual background required for the comparison. The implementation, configuration, and integration of each sampler into the common benchmarking framework are described in Chapter 3.

3 BENCHMARKING SAMPLING METHODS

This chapter presents the implementation of `mcmc-bench`, a common benchmarking framework integrating Random-Walk Metropolis-Hastings, `emcee`, `dynesty`, `PyMC Slice`, and `PyMC Sequential Monte Carlo`. The framework keeps the surrogate model, observations, prior distributions, likelihood, and quantification fixed while changing the sampler integration. All methods therefore target the same posterior and produce outputs in a common format, allowing their accuracy, convergence behavior, and sampling runtime to be evaluated consistently. The shared implementation is described first, followed by the sampler-specific integration details.

3.1 IMPLEMENTATION DETAILS

All benchmark runs are controlled through a single YAML configuration file, which defines the surrogate model, observations, calibrated parameters, prior distributions, likelihood, and sampler-specific settings. In the present benchmark, the Voellmy friction parameters μ and ξ are calibrated against the observed impact area using the uniform priors $\mu \sim \mathcal{U}(0.02, 0.50)$ and $\xi \sim \mathcal{U}(400, 4000)$. A Gaussian likelihood with a fixed noise standard deviation of $\sigma = 0.05$ is used; although optional noise calibration is supported, it is disabled for the current comparison. Changing the selected sampler therefore leaves the statistical calibration problem unchanged and modifies only the sampler and its configuration.

The pretrained RobustGaSP surrogate is exposed as an HTTP model using UM-Bridge. Each sampler connects to the same model server and sends candidate values of μ and ξ through this interface. Although the surrogate computes full predictive statistics, the model server exposes only the predictive mean, which enters the Gaussian likelihood on the sampler side.

```
output = self.model.predict(x)
# ScalarGaSP: (n, 4) -> [mean, lower, upper, std]
return [output[:, 0].tolist()]
# only column 0 = mean values are returned to the sampler
```

The server is configured with a single worker. Moreover, the exposed model currently provides function evaluations but no gradient information, which restricts the benchmark to gradient-free samplers.

The execution is orchestrated by a Nextflow workflow. It collects the surrogate and observation data, starts the UM-Bridge model server, and starts the process associated with the sampler selected in the configuration file. Each sampler runs in a separate Conda environment but receives the same configuration, observations, and model-server address. After sampling is complete, the model server is stopped and the generated outputs are passed to the common diagnostics and reporting processes.

To be sure that the same diagnostics and report-generation steps can be applied to every sampler, all implementations follow a common output contract. Each sampler run produces a posterior trace, chain-structured samples, log-probability values (when available), the measured sampling runtime, the number of surrogate-model evaluations, a corner plot, and an ArviZ InferenceData file. The InferenceData (iData) representation is then used to generate trace, posterior, and autocorrelation plots together with effective sample size (ESS) and R-hat statistics where applicable. These files, the run configuration, and the automatically generated report are collected in the session's output folder.

3.2 SAMPLER INTEGRATION

3.2.1 RANDOM-WALK METROPOLIS-HASTINGS

The RWMH integration uses the custom `rwmcmc` package and evaluates the common log-posterior directly. Each chain is initialized independently from the configured prior distributions and executed serially with a separate random-number generator. The proposal step sizes can be configured separately for μ and ζ , allowing their different numerical scales to be accommodated. After sampling, the configured burn-in period is removed, the retained chains are reshaped to the common output format, and an ArviZ InferenceData object is constructed explicitly for the shared diagnostics.

3.2.2 EMCEE

The `emcee` integration passes the same log-posterior function to an `EnsembleSampler`. Walker positions are initialized independently from the configured pri-

ors, while the number of walkers, number of sampling steps, and burn-in period are read from the common configuration. The package supports serial execution as well as thread- and process-based pools, although the present benchmark uses serial model evaluation. Following sampling, burn-in draws are removed and the emcee results are converted directly to the common arrays and ArviZ iData representation.

3.2.3 DYNESTY

Unlike the MCMC integrations, dynesty requires the likelihood and prior to be supplied separately. A dedicated prior transform maps points from the unit hypercube to the configured parameter distributions using their inverse cumulative distribution functions. The static nested sampler is controlled through the number of live points, the evidence-based stopping tolerance, and the number of worker processes. Because dynesty returns weighted samples, these are converted into an equally weighted trace for compatibility with the common plotting and diagnostics interface. The estimated log-evidence and its uncertainty are additionally retained in the sampler output.

3.2.4 PYMC SLICE AND SEQUENTIAL MONTE CARLO

Listing 3.1: PyTensor adapter used to expose the UM-Bridge likelihood to PyMC.

```
class UMBridgeLogLikelihood(Op):
    itypes = [pt.dvector]
    otypes = [pt.dscalar]

    def __init__(self, log_likelihood):
        self.log_likelihood = log_likelihood

    def perform(self, node, inputs, outputs):
        (parameters,) = inputs
        value = self.log_likelihood.eval(parameters)
        outputs[0][0] = np.asarray(value, dtype=np.float64)
```

Both PyMC integrations follow PyMC’s documented black-box likelihood pattern by using the numerical UM-Bridge likelihood to PyTensor through the operation shown in Listing 3.1 [19]. The operation accepts a “parameter” vector, evaluates the common likelihood through UM-Bridge, and returns a scalar log-likelihood. Since it

3 Benchmarking Sampling Methods

defines numerical evaluation but no gradient operation, it is used with gradient-free sampling methods. The configured priors are recreated as PyMC random variables, and the external likelihood is added to the model as a potential. Both methods return an iData object directly, from which the posterior samples are reshaped into the common output format.

4 EVALUATION

This chapter evaluates the five sampler integrations using the common Voellmy calibration problem. Chapter 3 addressed RQ1 by describing how the samplers were integrated into the modular benchmarking framework. The present chapter addresses RQ2 by defining criteria for accuracy, convergence, and computational efficiency and applying them to a common benchmark. Because the surrogate model, observations, priors, and likelihood are held fixed, the comparison focuses on differences from the sampling methods and their configured settings.

4.1 EXPERIMENTAL SETUP

All methods calibrate the Voellmy parameters μ and ξ against the observed impact-area value $y = 0.56139776$. The common statistical configuration uses the priors $\mu \sim \mathcal{U}(0.02, 0.50)$ and $\xi \sim \mathcal{U}(400, 4000)$ together with a Gaussian likelihood with fixed standard deviation $\sigma = 0.05$. The surrogate-model server uses one worker, and as a result, all sampler integrations are executed serially. Table 4.1 summarizes the sampler-specific settings and the number of posterior samples retained for analysis.

The samplers were not forced to produce identical numbers of stored samples because their computational controls are not directly equivalent. RWMH, emcee, and PyMC Slice use chain lengths, PyMC SMC uses a population size, and dynesty terminates according to an evidence-based stopping criterion. Matching the number of stored samples would therefore not imply an equal computational load. Instead, the target, surrogate-model server, and hardware were held fixed, while computational cost was recorded separately through sampling time and the number of surrogate-model evaluations.

The benchmark is executed on Ubuntu under WSL2 using an Intel Core i7-13700H processor with 20 logical processors and 15 GiB of memory available to WSL. Despite the available hardware parallelism, the surrogate server and sampler integrations were restricted to one worker, process, or core. The workflow used Nextflow 25.04.6. The sampler environments used Python 3.12.13 and ArviZ 0.20.0, together with

4 Evaluation

rwmcmc 0.1.0, emcee 3.1.6, PyMC 5.26.1 with PyTensor 2.35.1, and dynesty 3.0.0. The surrogate environment used psimpy 1.0.0, UM-Bridge 1.2.4, and RobustGaSP 0.6.4. Table 4.1 summarizes the sampler-specific settings.

Table 4.1: Sampler configurations used in the benchmark run.

Sampler	Configuration
RWMH	4 chains; 5000 steps per chain; 1000 burn-in steps; proposal scales (0.15, 1200)
emcee	8 walkers; 5000 steps per walker; 1000 burn-in steps; serial execution
PyMC Slice	4 chains; 1000 tuning steps and 4000 retained draws per chain; one core; random seed 42
PyMC SMC	4 chains; 2000 draws per chain; threshold 0.5; correlation threshold 0.01; one core; random seed 42
dynesty	500 live points; stopping tolerance $\Delta \log Z = 0.5$; one process

4.2 METRICS

The comparison considers three aspects: posterior agreement, convergence, and computational efficiency. Since an analytical reference posterior is not available for the calibration problem, accuracy is evaluated through agreement between the posterior approximations produced by the samplers. This agreement does not establish the absolute correctness of the posterior, since all methods share the same surrogate model and statistical assumptions, but it indicates whether the numerical methods approximate the common target consistently, or philosophically, mutually supporting hypotheses increase the credibility of a hypothesis.

Posterior agreement is evaluated by comparing the posterior mean and standard deviation of μ and ξ . The marginal histograms and joint corner plots are also inspected to compare the shapes and locations of the posterior distributions. Similar numerical summaries and distribution shapes indicate that the samplers approximate the common posterior consistently, whereas clear differences may indicate insufficient exploration or sensitivity to sampler settings.

For outputs represented by Markov chains, convergence is assessed using R-hat statistic and bulk and tail effective sample sizes (ESS). The R-hat statistic compares within-chain and between-chain variation, with values below 1.01 indicating that

no clear convergence problem has been detected. Bulk ESS describes the amount of information available for estimating the central region of the posterior, whereas tail ESS concerns the reliability of tail quantities and interval boundaries [28].

These diagnostics require different interpretations across the selected methods. The emcee walkers interact during sampling and are therefore not treated as independent chains for an R-hat decision in this study [9, 10]. Similarly, dynesty produces weighted nested-sampling points rather than multiple Markov chains, so R-hat is not applicable; its evidence-based stopping criterion and weighted posterior output instead describe termination of the algorithm [26]. PyMC SMC produces independent particle populations rather than conventional Markov chains, so its R-hat value is interpreted only as an indicator of agreement between repeated populations [6].

Because the final outputs of SMC and dynesty are not conventional Markov-chain outputs, their ArviZ ESS values are not interpreted as conventional convergence diagnostics.

Computational cost is measured using the sampling time and the number of successful surrogate-model evaluations. Model evaluations provide a hardware-independent indication of computational demand. Also they are particularly relevant when calibration requires repeated evaluations of an expensive forward model or its surrogate [31]. Sampling time additionally captures sampler and software overhead under the experimental environment. To relate computational costs to the posterior information, the benchmark uses the minimum bulk ESS across the calibrated parameters,

$$\text{ESS}_{\min} = \min(\text{ESS}_{\text{bulk},\mu}, \text{ESS}_{\text{bulk},\xi}),$$

and reports

$$\text{ESS per evaluation} = \frac{\text{ESS}_{\min}}{N_{\text{eval}}}, \quad \text{ESS per second} = \frac{\text{ESS}_{\min}}{t_{\text{sampling}}}.$$

Using the minimum parameter-specific ESS provides a conservative summary (worst-case-scenario) of each sampler's efficiency.

Table 4.2: Posterior summary statistics from the single benchmark run.

Sampler	μ : mean (SD)	ζ : mean (SD)
RWMH	0.1911 (0.0828)	2375.9 (955.9)
emcee	0.1867 (0.0862)	2310.1 (954.4)
PyMC Slice	0.1864 (0.0836)	2341.0 (957.7)
PyMC SMC	0.1868 (0.0828)	2354.8 (947.8)
dynesty	0.1773 (0.0852)	2312.6 (954.5)

4.3 RESULTS

4.3.1 POSTERIOR AGREEMENT

Table 4.2 compares the posterior mean and standard deviation obtained for the two calibrated parameters. Across the five samplers, the posterior mean ranges from 0.1773 to 0.1911 for μ and from 2310.1 to 2375.9 for ζ . The corresponding standard deviations are similar across the methods, ranging from 0.0828 to 0.0862 for μ and from 947.8 to 957.7 for ζ .

Figure 4.1 shows the summary of the result by showing the corner-plots for μ and ζ .

4.3.2 CONVERGENCE DIAGNOSTICS

Table 4.3 summarizes the most conservative diagnostics obtained for each sampler. The minimum bulk and tail ESS values and the maximum R-hat are taken across μ and ζ . The common diagnostic pipeline uses an ESS threshold of 400 and a maximum R-hat threshold of 1.01.

Table 4.3: Convergence diagnostics from the single benchmark run.

Sampler	Minimum bulk ESS	Minimum tail ESS	Maximum R-hat	Pipeline status
RWMH	1078.69	1666.16	1.0054	PASS
emcee	586.45	2556.16	–	N/A
PyMC Slice	8271.14	7631.33	1.0008	PASS
PyMC SMC	7447.42	6649.84	1.0002	PASS*
dynesty	1552.04	1486.68	–	N/A

RWMH and PyMC Slice satisfy both diagnostic thresholds for the present run. The emcee integration produces interacting walkers, so the pipeline does not use

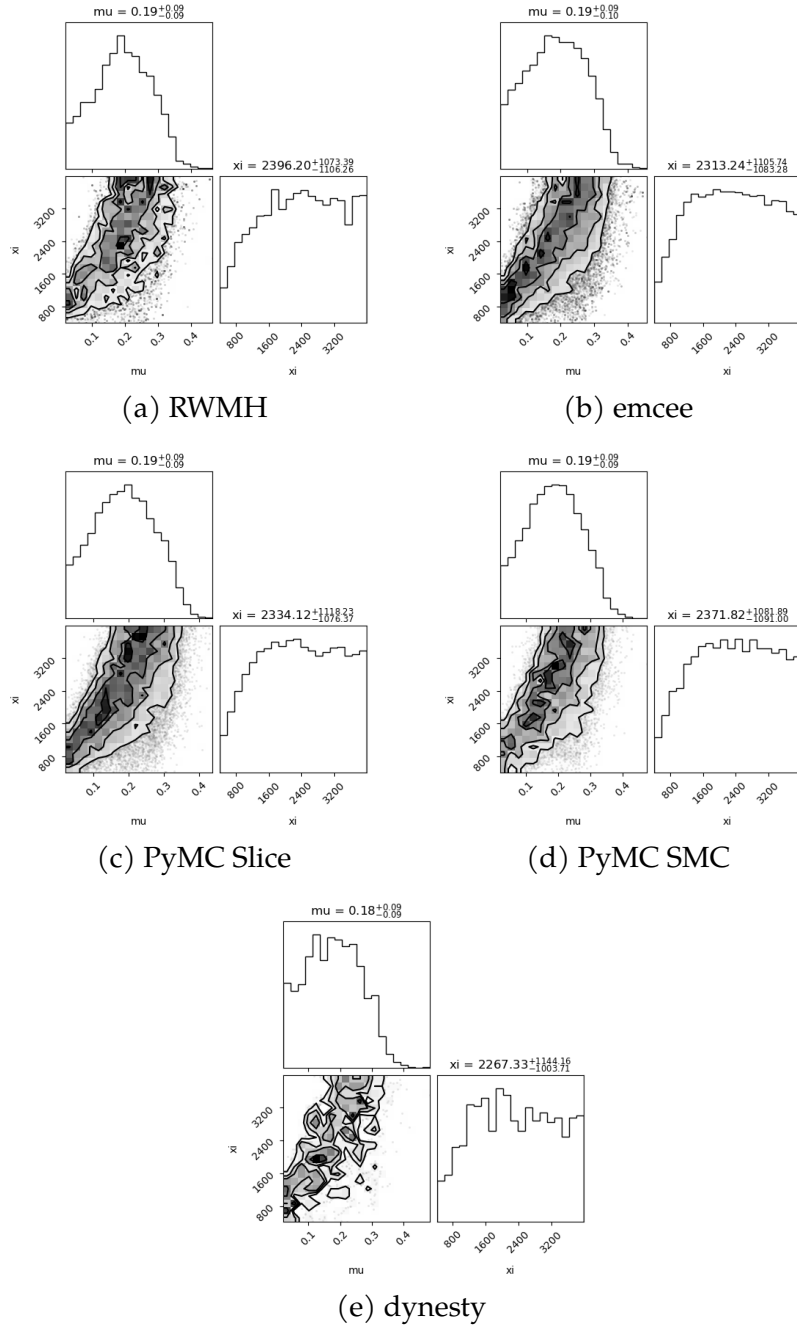


Figure 4.1: Joint posterior distributions obtained for (a) RWMH, (b) emcee, (c) PyMC Slice, (d) PyMC SMC, and (e) dynesty.

4 Evaluation

their R-hat values for a formal convergence decision. Dynesty likewise has no chain-based R-hat, its run terminated using the configured evidence tolerance of $\Delta \log Z = 0.5$ and produced an estimated log-evidence of 1.054 ± 0.074 .

The PyMC SMC output satisfies the numerical thresholds applied by the common pipeline. However, its sample groups are particle populations rather than classical Markov chains. Its PASS status therefore does not indicate agreement between Markov-chains. The ESS values reported for PyMC SMC and dynesty should similarly be interpreted as descriptive summaries of their equal-weight posterior outputs.

4.3.3 COMPUTATIONAL EFFICIENCY

Table 4.4 compares sampling time, surrogate-model evaluations, and the ESS results.

Table 4.4: Computational-efficiency results from the benchmark run.

Sampler	Evaluations	Time (s)	ESS/evaluation	ESS/s
RWMH	12039	28.13	0.0896	38.34
emcee	32587	74.05	0.0180	7.92
PyMC Slice	243362	516.52	0.0340	16.01
PyMC SMC	96000	209.34	0.0776	35.58
dynesty	3501	8.67	0.4433	178.93

In this run, dynesty required the fewest model evaluations and achieved the highest ESS per evaluation and ESS per second. PyMC Slice required the most evaluations and had the longest sampling time, while emcee produced the lowest normalized ESS values.

4.4 DISCUSSION

The similar posterior means and standard deviations indicate that the five implementations approximate the same general posterior region. RWMH and PyMC Slice also satisfy the chain-based convergence thresholds. The diagnostics for emcee, PyMC SMC, and dynesty require more careful interpretation because their walkers, particles, or weighted samples are not independent Markov chains.

dynesty was the most efficient method in this run, requiring the fewest model evaluations and the shortest sampling time. RWMH and PyMC SMC produced the

next-highest ESS per second, whereas emcee and PyMC Slice required more computational effort relative to their effective sample sizes. However, the ESS values for SMC and dynesty are just common-pipeline summaries and should not be compared directly with chain-based ESS values.

The conclusions are limited to one two-dimensional calibration problem, one run per sampler, and settings that were not exhaustively tuned. In addition, PyTensor used its Python fallback because a C++ compiler was unavailable, which may have increased the PyMC runtimes. The timing results therefore describe the present implementations and environment rather than the general performance of the sampler packages.

5 SUMMARY AND FUTURE WORK

This report developed and evaluated `mcmc-bench`, a common framework for comparing sampling methods in Bayesian calibration. Five gradient-free samplers were integrated through a shared Nextflow workflow and UM-Bridge model interface.

5.1 SUMMARY

Regarding RQ1, the implementation shows that sampler-specific environments can be integrated while keeping the surrogate model, statistical problem, and output format fixed. Regarding RQ2, posterior summaries, convergence diagnostics, model evaluations, and sampling time provide a practical comparison of the methods. The samplers produced similar posterior estimates in the mass-flow case study, while *dynesty* was the most computationally efficient in the single benchmark run.

5.2 FUTURE WORK

The evaluation should be repeated with multiple runs, more thoroughly tuned settings, higher-dimensional calibration problems, and alternative models. Extending the model interface with gradient information would enable gradient-based samplers, while additional model workers and sampler implementations in other programming languages would broaden the benchmark.

BIBLIOGRAPHY

1. J. Albert, C. Balázs, A. Fowlie, W. Handley, N. Hunt-Smith, R. Ruiz de Austri, and M. White. “A Comparison of Bayesian Sampling Algorithms for High-Dimensional Particle Physics and Cosmology Applications”. *Computer Physics Communications* 315, 2025, p. 109756. doi: [10.1016/j.cpc.2025.109756](https://doi.org/10.1016/j.cpc.2025.109756).
2. P.D. Arendt, D.W. Apley, and W. Chen. “Quantification of Model Uncertainty: Calibration, Model Discrepancy, and Identifiability”. *Journal of Mechanical Design* 134:10, 2012, p. 100908. doi: [10.1115/1.4007390](https://doi.org/10.1115/1.4007390).
3. M. Benning and M. Burger. “Modern Regularization Methods for Inverse Problems”. *Acta Numerica* 27, 2018, pp. 1–111. doi: [10.1017/S0962492918000016](https://doi.org/10.1017/S0962492918000016).
4. J. Brynjarsdóttir and A. O’Hagan. “Learning about Physical Parameters: The Importance of Model Discrepancy”. *Inverse Problems* 30:11, 2014, p. 114007. doi: [10.1088/0266-5611/30/11/114007](https://doi.org/10.1088/0266-5611/30/11/114007).
5. J. Buchner. “UltraNest: A Robust, General Purpose Bayesian Inference Engine”. *Journal of Open Source Software* 6:60, 2021, p. 3001. doi: [10.21105/joss.03001](https://doi.org/10.21105/joss.03001).
6. P. Del Moral, A. Doucet, and A. Jasra. “Sequential Monte Carlo Samplers”. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)* 68:3, 2006, pp. 411–436. doi: [10.1111/j.1467-9868.2006.00553.x](https://doi.org/10.1111/j.1467-9868.2006.00553.x).
7. P. Di Tommaso, M. Chatzou, E. W. Floden, P. Prieto Barja, E. Palumbo, and C. Notredame. “Nextflow Enables Reproducible Computational Workflows”. *Nature Biotechnology* 35:4, 2017, pp. 316–319. doi: [10.1038/nbt.3820](https://doi.org/10.1038/nbt.3820).
8. S. Duane, A. D. Kennedy, B. J. Pendleton, and D. Roweth. “Hybrid Monte Carlo”. *Physics Letters B* 195:2, 1987, pp. 216–222. doi: [10.1016/0370-2693\(87\)91197-X](https://doi.org/10.1016/0370-2693(87)91197-X).
9. D. Foreman-Mackey, D. W. Hogg, D. Lang, and J. Goodman. “emcee: The MCMC Hammer”. *Publications of the Astronomical Society of the Pacific* 125:925, 2013, pp. 306–312. doi: [10.1086/670067](https://doi.org/10.1086/670067).

10. J. Goodman and J. Weare. “Ensemble Samplers with Affine Invariance”. *Communications in Applied Mathematics and Computational Science* 5:1, 2010, pp. 65–80. DOI: [10.2140/camcos.2010.5.65](https://doi.org/10.2140/camcos.2010.5.65).
11. M. Gu. “RobustCalibration: Robust Calibration of Computer Models in R”. *The R Journal* 15:4, 2024, pp. 84–105. DOI: [10.32614/RJ-2023-085](https://doi.org/10.32614/RJ-2023-085).
12. W. K. Hastings. “Monte Carlo Sampling Methods Using Markov Chains and Their Applications”. *Biometrika* 57:1, 1970, pp. 97–109. DOI: [10.1093/biomet/57.1.97](https://doi.org/10.1093/biomet/57.1.97).
13. M. B. Heredia, N. Eckert, C. Prieur, and E. Thibert. “Bayesian Calibration of an Avalanche Model from Autocorrelated Measurements Along the Flow: Application to Velocities Extracted from Photogrammetric Images”. *Journal of Glaciology* 66:257, 2020, pp. 373–385. DOI: [10.1017/jog.2020.11](https://doi.org/10.1017/jog.2020.11).
14. M. D. Hoffman and A. Gelman. “The No-U-Turn Sampler: Adaptively Setting Path Lengths in Hamiltonian Monte Carlo”. *Journal of Machine Learning Research* 15, 2014, pp. 1593–1623.
15. M. C. Kennedy and A. O’Hagan. “Bayesian Calibration of Computer Models”. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)* 63:3, 2001, pp. 425–464. DOI: [10.1111/1467-9868.00294](https://doi.org/10.1111/1467-9868.00294).
16. L. Lannelongue, J. Grealey, and M. Inouye. “Green Algorithms: Quantifying the Carbon Footprint of Computation”. *Advanced Science* 8:12, 2021, p. 2100707. DOI: [10.1002/advs.202100707](https://doi.org/10.1002/advs.202100707).
17. F. D. Moura Neto and A. J. da Silva Neto. *An Introduction to Inverse Problems with Applications*. 1st ed. Springer, Berlin, Heidelberg, 2013. ISBN: 978-3-642-32557-1. DOI: [10.1007/978-3-642-32557-1](https://doi.org/10.1007/978-3-642-32557-1).
18. R. M. Neal. “Slice Sampling”. *The Annals of Statistics* 31:3, 2003, pp. 705–767. DOI: [10.1214/aos/1056562461](https://doi.org/10.1214/aos/1056562461).
19. M. Pitkin, J. Midtbø, O. Abril, and R. Vieira. “Using a “Black Box” Likelihood Function”. In: *PyMC Examples*. Ed. by PyMC Team. 2024. DOI: [10.5281/zenodo.5654871](https://doi.org/10.5281/zenodo.5654871). URL: https://www.pymc.io/projects/examples/en/latest/howto/blackbox_external_likelihood_numpy.html.

20. PyMC Development Team. `pymc.smc.sample_smc`. PyMC documentation. 2026. URL: https://www.pymc.io/projects/docs/en/stable/api/generated/pymc.smc.sample_smc.html (visited on 07/18/2026).
21. PyMC Development Team. `pymc.step_methods.hmc.NUTS`. PyMC documentation. 2026. URL: https://www.pymc.io/projects/docs/en/stable/api/generated/pymc.step_methods.hmc.NUTS.html (visited on 07/18/2026).
22. PyMC Development Team. `pymc.step_methods.Slice`. PyMC documentation. 2026. URL: https://www.pymc.io/projects/docs/en/stable/api/generated/pymc.step_methods.Slice.html (visited on 07/18/2026).
23. R. van de Schoot, S. Depaoli, R. King, B. Kramer, K. Märtens, M. G. Tadesse, M. Vannucci, A. Gelman, D. Veen, J. Willemsen, and C. Yau. “Bayesian Statistics and Modelling”. *Nature Reviews Methods Primers* 1:1, 2021, pp. 1–26. doi: [10.1038/s43586-020-00001-2](https://doi.org/10.1038/s43586-020-00001-2).
24. L. Seelinger, V. Cheng-Seelinger, A. Davis, M. Parno, and A. Reinartz. “UM-Bridge: Uncertainty Quantification and Modeling Bridge”. *Journal of Open Source Software* 8:83, 2023, p. 4748. doi: [10.21105/joss.04748](https://doi.org/10.21105/joss.04748).
25. J. Skilling. “Nested Sampling for General Bayesian Computation”. *Bayesian Analysis* 1:4, 2006, pp. 833–859. doi: [10.1214/06-BA127](https://doi.org/10.1214/06-BA127).
26. J.S. Speagle. “dynesty: A Dynamic Nested Sampling Package for Estimating Bayesian Posteriors and Evidences”. *Monthly Notices of the Royal Astronomical Society* 493:3, 2020, pp. 3132–3158. doi: [10.1093/mnras/staa278](https://doi.org/10.1093/mnras/staa278).
27. A. M. Stuart. “Inverse Problems: A Bayesian Perspective”. *Acta Numerica* 19, 2010, pp. 451–559. doi: [10.1017/S0962492910000061](https://doi.org/10.1017/S0962492910000061).
28. A. Vehtari, A. Gelman, D. Simpson, B. Carpenter, and P.-C. Bürkner. “Rank-Normalization, Folding, and Localization: An Improved $\hat{R}$ for Assessing Convergence of MCMC”. *Bayesian Analysis* 16:2, 2021, pp. 667–718. doi: [10.1214/20-BA1221](https://doi.org/10.1214/20-BA1221).
29. M.J. Williams, J. Veitch, and C. Messenger. “Nested Sampling with Normalizing Flows for Gravitational-Wave Inference”. *Physical Review D* 103:10, 2021, p. 103006. doi: [10.1103/PhysRevD.103.103006](https://doi.org/10.1103/PhysRevD.103.103006).

Bibliography

30. K.W.K. Wong, M. Gabri , and D. Foreman-Mackey. “flowMC: Normalizing Flow Enhanced Sampling Package for Probabilistic Inference in JAX”. *Journal of Open Source Software* 8:83, 2023, p. 5021. doi: [10.21105/joss.05021](https://doi.org/10.21105/joss.05021).
31. A. Yildiz, H. Zhao, and J. Kowalski. “Computationally-Feasible Uncertainty Quantification in Model-Based Landslide Risk Assessment”. *Frontiers in Earth Science* 10, 2023, p. 1032438. doi: [10.3389/feart.2022.1032438](https://doi.org/10.3389/feart.2022.1032438).
32. H. Zhao and J. Kowalski. “Bayesian Active Learning for Parameter Calibration of Landslide Run-Out Models”. *Landslides* 19:8, 2022, pp. 2033–2045. doi: [10.1007/s10346-022-01857-z](https://doi.org/10.1007/s10346-022-01857-z).
33. M.J. Zyphur and F.L. Oswald. “Bayesian Estimation and Inference: A User’s Guide”. *Journal of Management* 41:2, 2015, pp. 390–420. doi: [10.1177/0149206313501200](https://doi.org/10.1177/0149206313501200).